

Group 1 Capstone Project Report

Project Title

Shelf Product Detection, Classification, Counting, and Planogram-Aware Inventory Estimation using YOLO and CLIP

Team Details

- Team: Group 1
- Course: Capstone Project
- Institution: [Add Institution Name]
- Guide/Mentor: [Add Guide Name]
- Team Members: [Add Member 1], [Add Member 2], [Add Member 3], [Add Member 4]
- Submission Date: 07-Aug-2026

Abstract

Retail shelf monitoring is often performed manually, which is time-consuming and prone to errors. This project presents an AI-based system that automatically detects products in shelf images, classifies them into product labels, counts product facings, and estimates stock levels using a planogram. The solution combines a YOLO object detector with a CLIP-based image-text classifier and provides both command-line and Streamlit web interfaces for operational use. The implemented pipeline supports automatic label-profile selection (beverage vs shampoo), count export to CSV/JSON, visual annotation of detections, and inventory reconciliation with optional planogram-based quantity estimation. The system is designed to be practical for retail analytics workflows and can be extended to real-time video monitoring.

1. Introduction

Modern retail stores require frequent shelf audits to ensure product availability, reduce stockouts, and improve shopper experience. Manual audits involve visual counting and frequent data entry, leading to inconsistent accuracy. Computer vision offers an effective automation path by identifying products directly from images.

This capstone project develops a shelf product analytics system with the following goals:

- Detect shelf products from images.
- Classify each detected crop into product labels.
- Aggregate counts by category and product.
- Estimate stock and reorder need using planogram expectations.
- Provide an easy-to-use web interface for store operators.

2. Problem Statement

Given a shelf image, automatically estimate product presence and quantities while minimizing manual intervention. The system should output:

- Product-wise counts.
- Visual proof (annotated image).
- Inventory-friendly outputs (CSV/JSON).

- Reorder insights through planogram comparison.

3. Objectives

1. Build a robust object detection stage for shelf products.
2. Build a semantic classifier to map crops to known product labels.
3. Create a counting pipeline for single and batch image inference.
4. Build a user-facing Streamlit console with inventory and planogram tools.
5. Enable planogram-driven stock estimation and reorder flags.

4. System Overview and Architecture

The implemented workflow:

1. Input image is captured/uploaded.
2. YOLO model detects product bounding boxes.
3. Each crop is classified with CLIP against candidate prompts.
4. Predictions are grouped into Category | Product keys.
5. Counts are exported and displayed.
6. Optional planogram rules convert facings into estimated stock units.
7. Inventory is updated and reorder alerts are generated.

4.1 Core Modules

- ``app.py``: Command-line end-to-end inference (single image and batch mode).
- ``realtime_app.py``: Streamlit application with Detection Studio and Inventory Center.
- ``src/detector.py``: YOLO detector wrapper.
- ``src/classifier.py``: CLIP classifier with optional non-product rejection prompts.
- ``src/profile_selector.py``: Automatic profile selection between beverage and shampoo classifiers.
- ``src/pipeline.py``: Detection-to-classification orchestration and count summarization.
- ``src/planogram.py``: Planogram CRUD and quantity estimation logic.
- ``src/inventory.py``: Inventory load/save, adjustments, and reconciliation.
- ``src/visualize.py``: Annotated image generation.

5. Dataset and Model Strategy

5.1 Dataset

The training scripts are designed for SKU110K-style shelf data conversion to YOLO format. CSV annotations are converted into YOLO label files and a ``data.yaml`` configuration.

5.2 Detection Model

- Detector family: Ultralytics YOLO.
- Default deployed weights: ``Weights/best.pt``.
- Typical thresholds in this project:
- Confidence: 0.10

- IoU (NMS): 0.30

5.3 Classification Model

- Classifier: OpenAI CLIP (`openai/clip-vit-base-patch32`) via Hugging Face Transformers.
- Label strategy: curated prompt lists for beverage and shampoo domains.
- Additional quality filter: non-product prompts (price tags/signage) with rejection margin.

6. Methodology

6.1 Detection and Classification Pipeline

For each image:

1. Detect regions of interest (product boxes).
2. Crop each detection.
3. Run CLIP text-image matching against candidate prompts.
4. Select highest-probability product label (or reject non-product crop).
5. Aggregate final counts.

6.2 Auto Profile Selection

The system computes average confidence scores for each label profile (beverage/shampoo) and selects the profile with the higher mean confidence for that image.

6.3 Counting and Export

Counts are aggregated per key `Category | Product` and saved to:

- CSV: table format for reporting.
- JSON: structured format for downstream system integration.

6.4 Planogram Quantity Estimation

Each planogram item defines:

- expected facings
- full stock units
- reorder level

For each product:

- `detected\_facings = d`
- `expected\_facings = e`
- `compliance\_ratio = min(1, d/e)`
- `estimated\_units = round(compliance\_ratio \* full\_stock\_units)`
- `reorder\_qty = max(0, reorder\_level - estimated\_units)`
- `needs\_reorder = Yes if estimated\_units < reorder\_level else No`

This enables operational inventory approximation from visual shelf facings.

7. User Interface and Functional Features

The Streamlit app provides two operating screens.

7.1 Detection Studio

- Upload or capture shelf image.
- Run detection and classification.
- View input and annotated output side-by-side.
- Inspect profile scores and product counts.
- Optional inventory auto-sync from latest detection.

7.2 Inventory Center

- View/edit SKU stock and reorder levels.
- Add/deduct/reconcile stock from latest detection.
- Configure planogram items (expected facings, full stock units, reorder levels).
- Visual planogram diagram with facing fill state.
- Set stock directly from planogram estimate.
- Download inventory status as CSV.

8. Implementation Environment

- Language: Python 3.11
- UI: Streamlit
- Data handling: Pandas, JSON, CSV
- Vision/Image: PIL (Pillow)
- Detection: Ultralytics YOLO
- Classification: Transformers CLIP + Torch

9. Observed Output Snapshot

From current sample result on `cooldrinks.jpg`, totals include:

- Soft Drinks | Coca-Cola: 17
- Soft Drinks | Fanta: 6
- Soft Drinks | Sprite: 5
- Soft Drinks | 7UP: 2

This confirms successful end-to-end operation: detect -> classify -> count -> export.

10. Strengths of the Solution

1. End-to-end pipeline from image input to inventory-ready outputs.
2. Practical dual operation modes (CLI and web UI).
3. Auto profile selection increases label relevance across shelf types.
4. Planogram integration converts visual counts into stock estimates.

5. Human-in-the-loop corrections architecture supports iterative quality improvement.

11. Limitations

1. Performance depends on detector quality under occlusion and motion blur.
2. CLIP prompt-based classification may confuse visually similar brands.
3. Planogram estimate is proportional and does not model shelf depth/back stock.
4. No direct tracker yet for robust frame-wise counting in video streams.
5. Formal quantitative evaluation metrics (mAP / top-k) are not fully benchmarked in this report.

12. Future Enhancements

1. Add temporal tracking for real-time camera streams.
2. Train/fine-tune brand-specific classifiers with local store datasets.
3. Add confidence-based unknown class handling.
4. Extend planogram logic with row/slot position constraints.
5. Integrate dashboards for trend analysis and automated replenishment alerts.

13. Conclusion

This capstone demonstrates a deployable AI pipeline for shelf product intelligence. By combining YOLO detection, CLIP-based classification, and planogram-aware stock estimation, the project reduces manual shelf audit effort and provides actionable inventory insights. The implemented system is modular, extensible, and suitable as a foundation for production-grade retail analytics.

14. References

1. Ultralytics YOLO Documentation.
2. Radford et al., CLIP: Learning Transferable Visual Models From Natural Language Supervision.
3. SKU110K Dataset paper and benchmark resources.
4. Streamlit Documentation.
5. Hugging Face Transformers Documentation.

15. Appendix

15.1 Example Commands

- Single-image inference:
- ``python app.py --image cooldrinks.jpg --detector-weights Weights/best.pt --output-image outputs/annotated_result.jpg``
- Streamlit app:
- ``streamlit run realtime_app.py``

15.2 Expected Deliverables

- Annotated output image(s)
- Product count CSV and JSON
- Inventory snapshot CSV

- Planogram configuration JSON